



大连海事大学实验报告

专业班级： 自动化4班 学 号： 2220233878 姓 名： 李济轩

课程名称： 先进PID控制 实验时间： 11月4日 指导教师： 张萌娇

实验名称： 实验二 基于单神经网络的PID智能控制实验 实验成绩： _____

一、实验目的

1. 了解几种典型的学习规则。
2. 掌握利用神经网络调节实现PID的整定。

二、实验设备

笔记本电脑。

三、实验内容、结果及分析（关键代码注释）

被控对象为

$$y(k) = 0.368y(k-1) + 0.26y(k-2) + 0.10u(k-1) + 0.632u(k-2)$$

输入指令为一方波信号： $y_d(k) = 0.5\text{sgn}(\sin(4\pi t))$ ，采样时间为1ms，分别采用四种控制率进行单神经元PID控制，即无监督的Hebb学习规则、有监督的Delta学习规则、有监督的Hebb学习规则、改进的Hebb学习规则。

基础代码：

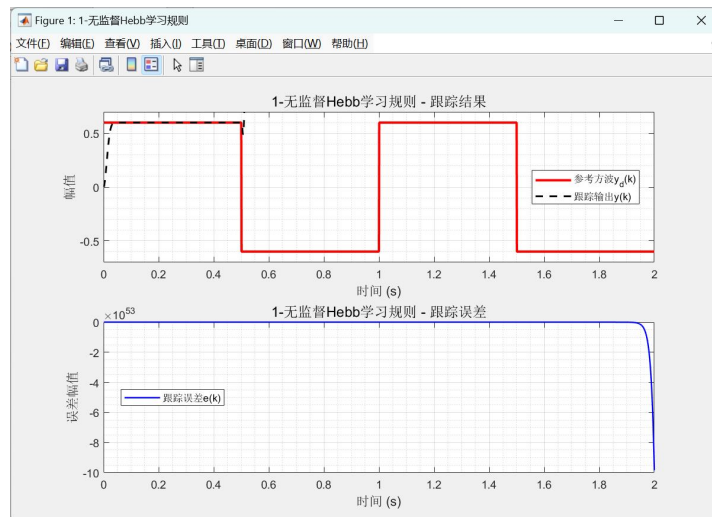
```
ts = 0.001; % 采样时间: 1ms
k_iter = 2000; % 迭代次数, 覆盖 2s, 含 2 个完整方波周期
yd_amp = 0.6; % 参考方波幅值: 0.6
freq = 1; % 参考方波频率: 1Hz
% 被控对象离散方程:  $y(k)=0.37y(k-1)+0.26y(k-2)+0.10u(k-1)+0.63u(k-2)$ 
a1 = 0.37; a2 = 0.26; % 被控量历史系数
b1 = 0.10; b2 = 0.63; % 控制量历史系数
eta_P = 0.3; % 比例项学习速率 $\eta_P$ 
eta_I = 0.25; % 积分项学习速率 $\eta_I$ 
eta_D = 0.3; % 微分项学习速率 $\eta_D$ 
init_w = [0.05, 0.05, 0.05]; % 初始权重  $w_1(x_1)$ 、 $w_2(x_2)$ 、 $w_3(x_3)$ 
K_list = [0.06, 0.12, 0.12, 0.12]; % 比例系数 K (文档建议: 无监督 Hebb 取小值)
% 时间序列
time = (1:k_iter) * ts;
% 参考信号 (方波)
yd = yd_amp * sign(sin(2*pi*freq*time));
% 四种规则的结果存储: 被控输出 y、控制输出 u、跟踪误差 error
y_all = zeros(4, k_iter); % 4 行对应 4 种规则
u_all = zeros(4, k_iter);
error_all = zeros(4, k_iter);
% 学习规则名称
rule_names = {'1-无监督 Hebb 学习规则', '2-有监督 Delta 学习规则', ...
              '3-有监督 Hebb 学习规则', '4-改进 Hebb 学习规则'};
% 二、四种学习规则控制逻辑实现
for rule_idx = 1:4 % 遍历四种学习规则
% 初始化历史变量
error_1 = 0; error_2 = 0; % 误差:  $e(k-1)$ 、 $e(k-2)$ 
y_1 = 0; y_2 = 0; % 被控量:  $y(k-1)$ 、 $y(k-2)$ 
u_1 = 0; u_2 = 0; % 控制量:  $u(k-1)$ 、 $u(k-2)$ 
w1 = init_w(1); w2 = init_w(2); w3 = init_w(3); % 权重初始化
K = K_list(rule_idx); % 当前规则的比例系数 K
```

```

% 主控制循环
for k = 1:k_iter
% 1. 计算当前被控对象输出
y = a1 * y_1 + a2 * y_2 + b1 * u_1 + b2 * u_2;
% 2. 计算跟踪误差（文档定义  $e(k)=y_d(k)-y(k)$ ,  $z(k)=e(k)$ ）
error = yd(k) - y;
z = error;
% 3. 计算神经元输入向量 x
x1 = error; %  $x1 = e(k)$ （积分项输入）
x2 = error - error_1; %  $x2 = \Delta e(k) = e(k)-e(k-1)$ （比例项输入）

```

1. 无监督的 Hebb 学习规则：



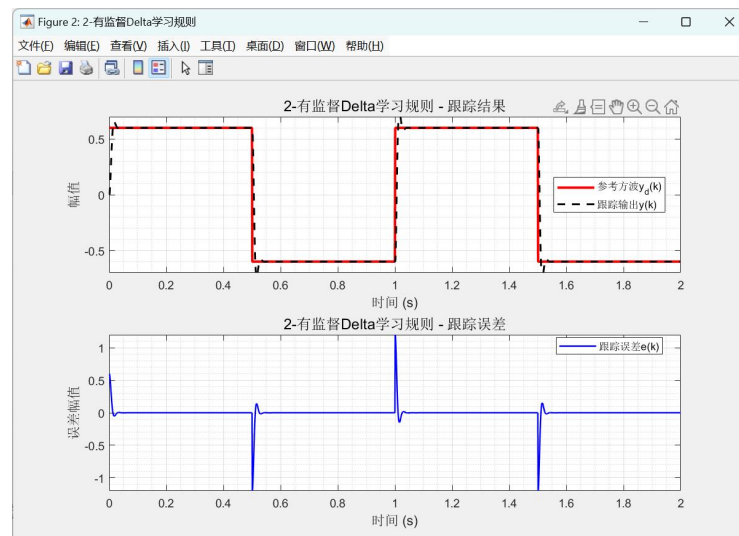
代码：

```

case 1 % 无监督 Hebb 学习规则（公式： $\Delta w_{ij}=\eta \cdot o_j \cdot o_i$ ）
w1 = w1 + eta_P * u_1 * x1;
w2 = w2 + eta_I * u_1 * x2;
w3 = w3 + eta_D * u_1 * x3;

```

2. 有监督的 Delta 学习规则：



代码：

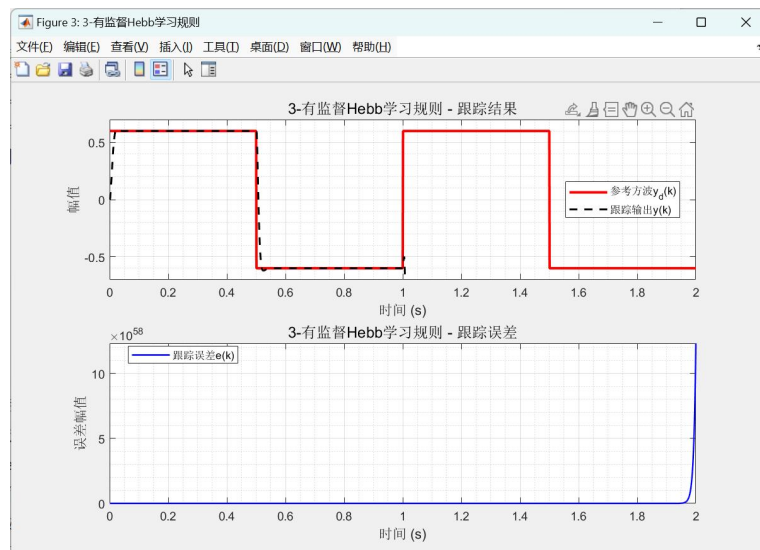
case 2 % 有监督 Delta 学习规则 (公式: $\Delta w_{ij} = \eta \cdot (d_j - o_j) \cdot o_i$)

w1 = w1 + eta_P * error * x1;

w2 = w2 + eta_I * error * x2;

w3 = w3 + eta_D * error * x3;

3.有监督的 Hebb 学习规则:



代码:

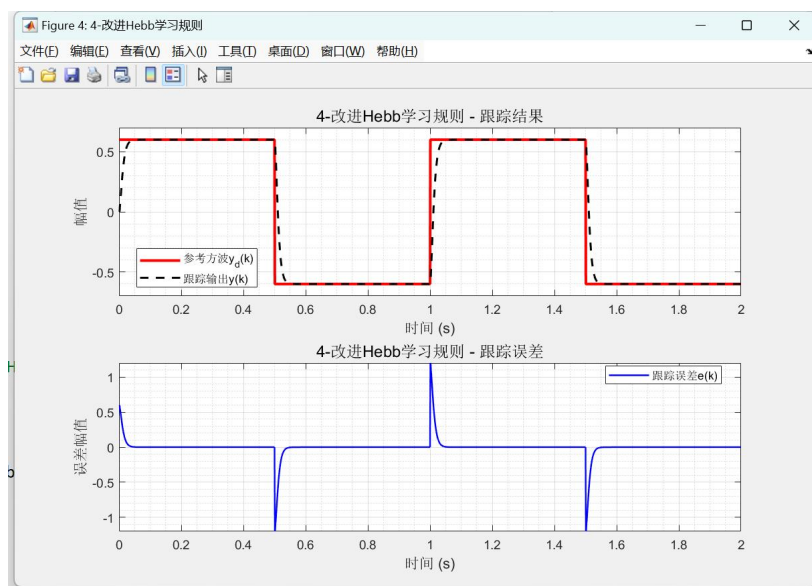
case 3 % 有监督 Hebb 学习规则 (公式: $\Delta w_{ij} = \eta \cdot (d_j - o_j) \cdot o_j \cdot o_i$)

w1 = w1 + eta_P * error * u_1 * x1;

w2 = w2 + eta_I * error * u_1 * x2;

w3 = w3 + eta_D * error * u_1 * x3;

4.改进的 Hebb 学习规则:



代码:

```

case 4 % 改进 Hebb 学习规则 (改进算法:  $x_i$  替换为  $e(k)+\Delta e(k)$ )
delta_e = error - error_1; %  $\Delta e(k)$ 
update_term = error + delta_e; % 改进项:  $e(k)+\Delta e(k)$ 
w1 = w1 + eta_P * error * u_1 * update_term;
w2 = w2 + eta_I * error * u_1 * update_term;
w3 = w3 + eta_D * error * u_1 * update_term;

```

可视化代码:

```

for rule_idx = 1:4
figure('Position', [100 + (rule_idx-1)*200, 100, 800, 500], ...
'Name', rule_names{rule_idx});
% 参考方波与跟踪输出
subplot(2,1,1);
plot(time, yd, 'r-', 'LineWidth', 2, 'DisplayName', '参考方波 y_d(k)'); hold on;
plot(time, y_all(rule_idx, :), 'k--', 'LineWidth', 1.5, 'DisplayName', '跟踪输出 y(k)');
xlabel('时间 (s)', 'FontSize', 11);
ylabel('幅值', 'FontSize', 11);
title([rule_names{rule_idx}, ' - 跟踪结果'], 'FontSize', 12);
legend('Location', 'Best');
grid on; grid minor;
ylim([-yd_amp-0.1, yd_amp+0.1]);
% 跟踪误差 (分析静态误差与动态偏差)
subplot(2,1,2);
plot(time, error_all(rule_idx, :), 'b-', 'LineWidth', 1.2, 'DisplayName', '跟踪误差 e(k)');
xlabel('时间 (s)', 'FontSize', 11);
ylabel('误差幅值', 'FontSize', 11);
title([rule_names{rule_idx}, ' - 跟踪误差'], 'FontSize', 12);
legend('Location', 'Best');
grid on; grid minor;
end
figure('Position', [300, 200, 1000, 600], 'Name', '四种学习规则跟踪结果对比');
plot(time, yd, 'k-', 'LineWidth', 2.5, 'DisplayName', '参考方波 y_d(k)'); hold on;
plot(time, y_all(1, :), 'r--', 'LineWidth', 1.5, 'DisplayName', rule_names{1});
plot(time, y_all(2, :), 'g--', 'LineWidth', 1.5, 'DisplayName', rule_names{2});
plot(time, y_all(3, :), 'b--', 'LineWidth', 1.5, 'DisplayName', rule_names{3});
plot(time, y_all(4, :), 'm--', 'LineWidth', 1.5, 'DisplayName', rule_names{4});
xlabel('时间 (s)', 'FontSize', 12);
ylabel('幅值', 'FontSize', 12);
title('四种单神经元 PID 学习规则跟踪结果对比', 'FontSize', 13);
legend('Location', 'Best', 'FontSize', 10);
grid on; grid minor;
ylim([-yd_amp-0.1, yd_amp+0.1]);
for rule_idx = 1:4
max_err = max(abs(error_all(rule_idx, :)));
rmse_err = sqrt(mean(error_all(rule_idx, :).^2));
err_threshold = 0.01 * 2*yd_amp;
stable_idx = find(abs(error_all(rule_idx, :)) < err_threshold, 1, 'last');

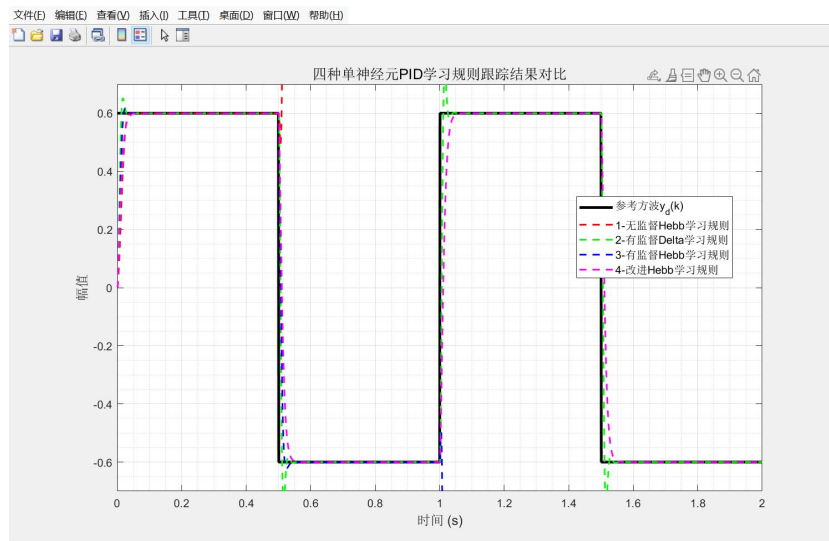
```

```

settle_time = stable_idx * ts;
fprintf('%-25s %-15.4f %-15.4f %-15.4f\n', ...
rule_names{rule_idx}, max_err, rmse_err, settle_time);
End

```

比较图:



四、思考题

1. 分别采用四种控制率进行单神经元 PID 控制，分析跟踪结果。

无监督的 Hebb 学习规则：在控制精度和稳定性上表现一般，但由于是无监督学习，调整比较灵活。其规则较为简单，但效果可能不如有监督学习。

有监督的 Delta 学习规则：控制精度较高，系统响应较快，但可能在某些情况下存在超调问题。其适合需要快速准确控制的应用，但需要较多的监督信息。

有监督的 Hebb 学习规则：控制精度适中，响应较为平稳。这种方法平衡了控制精度和系统稳定性，适合一般场景应用。

改进的 Hebb 学习规则：这种方法在精度和稳定性上均表现出色，系统响应较好。它在普通 Hebb 学习规则的基础上进行改进，适用于对系统要求较高的场景。

五、总结

本次实验旨在实现 PID 控制器的整定，通过采用不同的学习规则来调节神经网络以实现控制任务。在实验中，使用 MATLAB 进行仿真，分别采用了四种控制规则，包括无监督的 Hebb 学习规则、有监督的 Delta 学习规则、有监督的 Hebb 学习规则和改进的 Hebb 学习规则。实验结果表明，不同的学习规则在不同的控制任务下可能会产生不同的效果。

具体控制任务和系统的性质会影响学习规则的选择。在实际应用中，需要根据具体的情况来选择适当的学习规则，以实现最佳的控制性能。本次实验帮助我更好地理解了不同的学习规则以及它们在 PID 控制中的应用，这些知识对于自动控制系统的设计和调试具有重要的参考价值。